

Herald



CLAUDE.md

Field	Value
Revision	10
Created	2026-05-15
Last modified	2026-05-22
Status	active
Status summary	r10: Wave 6 (pherald inbound runtime + Claude Code headless bridge — code-doc closure; tag v0.4.0 deferred to T13b post-live-evidence). New pherald listen Cobra subcommand drives the closed-loop runtime: Telegram getUpdates long-poll + bot self-filter (§32.9 anti-echo via bot.Me.Username) + OnPhoto/OnDocument/OnVoice handlers with sha256 content-addressed attachment download (~/.herald/inbox/<sha>.<ext>, idempotent) + Claude Code dispatch with Opus pinned in argv (claude --model claude-opus-4-7, §33.7) + envelope pre-text — verbatim operator wording "We have received new message from our communication channel ..." preceding the existing <<<HERALD-DISPATCH-v1>>> block (§33.6) + <<<HERALD-REPLY>>> parser with action routing (reply / issue.open / event.emit) + tgram.SendReply wiring reply_to_message_id. T10a --qa-out-dir JSONL journaling (§107.x). 12 Wave-6 commits (T1=ad87d7f → T12=96c7c6b). 8 new e2e invariants E63-E70 (currently SKIP-with-documented-reason — convert to PASS once T10b lands operator-supplied live evidence). Wave 6 mutation gate tests / test_wave6_mutation_meta.sh (3 paired hermetic mutations, 4/4 PASS). HRD-100 atomic Issues→Fixed. Spec V3 r11→r12 with §32.9 + §33.6 + §33.7 + §43 pherald listen row. Workspace at 16 modules (Wave 5 added qaherald; Wave 6 added NO new module — inbound is pherald / internal / inbound /). Prior r9: Wave 5 T1 qaherald module skeleton landed (16th workspace module + 8th flavor binary). Prior r8: Wave 4b TOON status pointer; tag v0.3.0 substrate. Prior r5/r6: §107 End-user-usability covenant section restating the verbatim operator mandate at Herald level + ToC entry; ties to HERALD_CONSTITUTION.md §107 + inheritance-gate invariant I8a (paired with I8b/c on AGENTS.md + HERALD_CONSTITUTION.md).
Issues	none
Issues summary	—
Fixed	

Field	Value
Fixed summary	spec-path references (r2), pre-implementation-language update (r3), submodules + HRD-docs + codegraph-index enumeration (r4), §107 mandate restatement + I8a anchor (r5), Wave-2/3a workspace-module-count refresh (r7), Wave 4a + 4b status pointer + e2e/mutation tally refresh (r8), Wave 5 T1 qaherald skeleton + commons_tls enumeration fold-in (r9), Wave 6 inbound runtime + CC headless bridge code-doc closure (r10) aligned with HRD-009/HRD-009b/HRD-013/HRD-014 landing in the same commit; r4 closes the discoverability gaps observed during a fresh /init review; r5 closes the Herald-level explicit-restatement gap identified by the 2026-05-20 audit; r7 closes the doc drift observed during the Wave 3a final review; r8 records the Wave 4a→4b transport/wire-format substrate upgrades; r9 lands Wave 5 T1 (qaherald/cmd/qaherald + commons/branding.go qa flavor) and back-fills the Wave 4a commons_tls workspace enumeration that was missing from r7/r8; r10 lands Wave 6 — pherald inbound runtime (pherald listen Cobra subcommand, pherald/internal/inbound/Dispatcher with <<<HERALD-REPLY>>> action routing, tgram bot self-filter + OnPhoto/OnDocument/OnVoice with sha256 content-addressed attachment download, claude_code dispatcher pinned to Opus with verbatim pre-text envelope, tgram.SendReply with reply_to_message_id, T10a --qa-out-dir JSONL journaling); spec V3 r11→r12 + HRD-100 atomic Issues→Fixed; tag v0.4.0 deferred to T13b (post-T10b live evidence).
Continuation	bump again when T10b lands operator-supplied live evidence under docs/qa/HRD-100-<run-id>/ and T13b tags v0.4.0; then continue with Wave 7 (genericize messenger-channel framework per operator mandate 2026-05-22 — Slack/Max/Email next), Wave 3c carry-over (HRD-024 iherald paging, HRD-033 destructive-guard body, remaining HRD-018..028 constitution bindings), §43 command catalogue HRD-029.056, comprehensive docs audit (task #147), and the OOM-Protect Herald flavor (oherald, future post-Wave-6 closure per operator 2026-05-22).

Table of contents

- [INHERITED FROM Helix Constitution \(parent-discovery\)](#)
- [Project status](#)
- [End-user-usability covenant \(Herald §107 / Helix §11.4 — MANDATORY ANTI-BLUFF\)](#)
- [Mission \(from the spec\)](#)
- [Intended stack](#)
- [Multi-host mirror convention](#)
- [Inheritance gate \(run before any commit touching root docs\)](#)
- [Spec-change rule \(load-bearing — docs/specs/mvp/specification.V3.md §"Specification documents"\)](#)
- [Project conventions from the spec \(apply when scaffolding\)](#)
- [constitutable/ directory \(parent-project extension hook\)](#)
- [Documentation artefacts \(PDF/HTML siblings\)](#)
- [Notes for future scaffolding](#)

This file provides guidance to Claude Code (claude.ai/code) when working with code in this repository.

INHERITED FROM Helix Constitution (parent-discovery)

Herald is consumed as a submodule of a parent project that already carries the Helix Constitution submodule at <parent>/constitution/. Herald therefore does **NOT** keep its own copy. Locate the constitution from any nested depth by walking up parents until you find <ancestor>/constitution/Constitution.md, or by running the canonical helper:

```
CONST_DIR="$(bash "$(find . -type d -name constitution -print  
-quit 2>/dev/null)/find_constitution.sh)"  
# or, more robustly, from any starting directory:  
CONST_DIR="$(bash <ancestor>/constitution/find_constitution.sh)"
```

For standalone development of Herald (no parent project), clone the constitution alongside Herald:

```
git clone git@github.com:HelixDevelopment/HelixConstitution.git \  
$(dirname "$PWD")/constitution
```

Once located, all rules in <discovered>/CLAUDE.md and the <discovered>/Constitution.md it references apply unconditionally. Herald-specific rules below extend them — they **MUST NOT** weaken any inherited rule. When this file disagrees with the discovered constitution, the constitution wins.

Canonical: <https://github.com/HelixDevelopment/HelixConstitution>

Read order on a cold start:

1. <discovered-constitution>/CLAUDE.md + Constitution.md — universal Helix rules. Inherited unconditionally.
2. <discovered-constitution>/AGENTS.md — agent guardrails (anti-bluff, no-guessing, paired mutations).
3. README.md — Herald overview + how-to.
4. This file (Herald-specific notes below).
5. docs/guides/HERALD_CONSTITUTION.md — Herald's project-specific constitutional extensions.
6. docs/guides/CONSTITUTION_INHERITANCE.md — operator/agent guide for the discovery contract + the inheritance gate.
7. docs/specs/mvp/specification.V3.md — mission spec (mostly TBD).

Project status

Herald is in **first-implementation cycle (r1)** as of 2026-05-20. The Go scaffold has landed; live-integration work (HRD-010..HRD-012, HRD-016) is still open. The repo contains:

- README.md — mission, deployment model, inheritance contract, quickstart.
- docs/specs/mvp/specification.V3.md — MVP spec stub (substantive sections TBD).
- docs/guides/HERALD_CONSTITUTION.md — Herald's project constitution (extends Helix).
- docs/guides/CONSTITUTION_INHERITANCE.md — operator/agent guide for parent-discovery + gate semantics.
- upstreams/ — Herald's mirror declarations (see below).

- `tests/test_constitution_inheritance.sh + _meta.sh` — paired inheritance gate (§1.1).
- `.gitignore` tuned for Go + `.DS_Store`.

Herald does **not** ship a `constitution/` submodule of its own; the parent project provides it. See `docs/guides/CONSTITUTION_INHERITANCE.md`.

As of 2026-05-20 the Go scaffold has landed (first-implementation cycle r1). The repo now contains:

- `go.work` (gitignored locally; check in if the project wants reproducible workspaces — current convention: gitignored per spec §9.1).
- `commons/` (L0) — `commons/types.go` with the full §11.0 Channel contract + `Subscriber/CloudEventEnvelope/TraceContext/Branding/ChannelID`; `commons/clock.go` Clock abstraction; `commons/uuidv7.go`; `commons/branding.go` per-flavor factory.
- `commons_prefix/` — §8.2 3-letter prefix algorithm.
- `commons_messaging/channels/null/` — full §11.14 `null://` sandbox adapter (working, tested).
- `commons_messaging/channels/tgram/` — Telegram adapter SCAFFOLD (HRD-011 open).
- `commons_messaging/dispatch/claude_code/` — Claude Code session-resolution + envelope formatter; `live claude --resume` invocation pending (HRD-012 open).
- `commons_storage/` — 9 SQL migrations (000001..000005) embedded via `// go:embed`; `pgx` + `River` + `Redis` wiring pending (HRD-010 open).
- `pherald/cmd/pherald/` — Cobra CLI; `pherald version` works end-to-end; other subcommands stubbed with HRD-NNN error pointers.
- `quickstart/` — Herald-specific Docker/Podman Compose + Dockerfile + `otel-config + .env.example` per §26.5 (HRD-008 open for operator validation). Migrated from `containers/quickstart/` 2026-05-20 when the `containers/` submodule was added.
- `containers/` — git submodule pointing at `git@github.com:vasic-digital/containers.git` (the `digital.vasic.containers` Go module — runtime auto-detection + on-demand container boot + lifecycle/health). Imported by Foundation tests + the `pherald doctor` subcommand to bring Postgres + Redis + OTel up on-demand.

Build + test: from the repo root:

```
go build ./commons/... ./commons_prefix/... ./
    commons_messaging/... ./commons_storage/... ./
    commons_constitution/... ./commons_infra/... ./
    pherald/...
go test -race -count=1 ./commons/... ./commons_prefix/... ./
    commons_messaging/... ./commons_storage/... ./
    commons_constitution/... ./commons_infra/... ./
    pherald/...
```

Tests pass on Go 1.25+ (verified on 1.26). Workspace is configured via `go.work` listing **16** Herald modules: the 7 foundation modules (`commons`, `commons_constitution`, `commons_infra`, `commons_messaging`, `commons_prefix`, `commons_storage`, `pherald`) plus `commons_auth` (Wave 3a JWT verifier / Gin middleware), `commons_tls` (Wave 4a auto-cert loader for the dual HTTP/3+TLS listener), and the 7 flavor binaries: the 6 Wave 2 flavors (`sherald`, `cherald`, `bherald`, `rherald`, `iherald`, `scherald`) plus

qaherald (Wave 5 — QA bot, pherald  Telegram round-trip automation per §107.x docs/qa evidence mandate). `go.work` itself is gitignored per spec §9.1; a fresh clone needs `go work init && go work use ./...` (or copy the existing snippet from the project's local working tree). The 9 Helix-stack submodules under `submodules/` are referenced via `replace` directives in the consuming modules' `go.mod`, not via `go.work`.

Anti-bluff verification (run before any release tag or risky commit):

```
scripts/audit_antibluff.sh      # 3 invariants: §11.4 anchor +
                                tests + paired meta
scripts/codegraph_validate.sh   # CodeGraph index integrity (7
                                probes)
scripts/e2e_bluff_hunt.sh       # 14 end-to-end checks against
                                real services
```

CodeGraph index lives in `.codegraph/` (`codegraph.db + config.json`, both gitignored). Rebuild with `scripts/codegraph_setup.sh` when source layout changes; `codegraph_validate.sh` will FAIL otherwise.

`scripts/e2e_bluff_hunt.sh` is the canonical end-to-end smoke per Universal §11.4. It builds pherald, runs the full test suite, starts a real Gin server + hits every `/v1` route + asserts response bodies, boots a real Postgres container via the `containers/` submodule, runs the M2 integration tests against it, and graceful-shutdowns. ALL 14 invariants must PASS — a single FAIL means a real feature is broken for end users. At 2026-05-20 it reports 14/14 PASS.

When the user asks to "add a feature" the spec is the source of truth — find the relevant §, then the relevant module + package, then the relevant HRD-NNN if one is already open. New work opens a new HRD-NNN in `docs/Issues.md` per V3 §8.3 lifecycle.

Do not invent build/test commands beyond what `go test ./<module>/...` provides. Live-integration tests (Telegram bot, Claude Code session, real Postgres) require operator-supplied credentials — see `docs/CONTINUATION.md` for the live-test handoff prompt.

End-user-usability covenant (Herald §107 / Helix §11.4 — MANDATORY ANTI-BLUFF)

Forensic anchor — verbatim operator mandate:

"all existing tests and Challenges do work in anti-bluff manner - they MUST confirm that all tested codebase really works as expected! We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product! This MUST BE part of Constitution of our project, its CLAUDE.MD and AGENTS.MD if it is not there already, and to be applied to all Submodules's Constitution, CLAUDE.MD and AGENTS.MD as well (if not there already)!"

The bar for shipping any Herald feature is **NOT** "tests pass" — it is **"the end user of the flavor binary can actually use the feature."** Every PASS (unit, integration, gate, Challenge, smoke, e2e) MUST carry positive runtime evidence that the user-visible behaviour works. Metadata-only / configuration-only / "absence-of-error" / grep-only PASS are §11.4 PASS-bluffs and constitute critical defects regardless of how green the summary line looks. Canonical Herald

authority: docs/guides/HERALD_CONSTITUTION.md §107. Canonical Helix authority: <discovered>/Constitution.md §11.4 + §11.4.1..§11.4.16 and <discovered>/CLAUDE.md "MANDATORY ANTI-BLUFF COVENANT — END-USER QUALITY GUARANTEE". Canonical Herald evidence: scripts/e2e_bluff_hunt.sh (14 invariants against real services; ALL must PASS). Inheritance gate invariant **I8a** asserts this covenant anchor is present in this file.

§107.x — docs/qa/ Evidence Mandate (operator mandate, 2026-05-22; cascades from Helix §11.4.83)

Forensic anchor — verbatim operator mandate (2026-05-22):

"every feature that ships MUST carry a recorded e2e communication transcript + any attached materials under docs/qa/<run-id>/ (per-feature subdirectories). A feature with no QA transcript is itself a §107 PASS-bluff — it claims to work but has no auditable runtime evidence. Bot-driven automation (e.g. Herald's planned qaherald binary) MUST preserve full bidirectional communication threads as proof."

Every Herald feature that ships — every flavor binary (pherald, sherald, cherald, bherald, rherald, iherald, scherald, future qaherald) and every /v1/* route they expose — MUST carry a recorded end-to-end communication transcript plus all attached materials (Telegram screenshots, Gin response bodies in JSON or TOON, OpenTelemetry trace exports, container logs) committed under docs/qa/<run-id>/. A Herald feature that ships without a docs/qa/<run-id>/ directory is by definition a §107 PASS-bluff — its e2e_bluff_hunt PASS line claims it works for end users, but no auditable runtime evidence proves an end user ever exercised it.

Operative rule for Herald. (1) Every Herald HRD-NNN (V3 §8.3) work-item that introduces a user-visible feature MUST land its docs/qa/HRD-NNN/ directory (timestamp-prefixed if multiple runs) in the same logical work effort. (2) Bidirectional transcripts only — for Telegram-driven features (HRD-011, planned qaherald), capture both directions of the bot conversation; for Gin /v1 routes, capture both the request payload and the full response body. (3) Attached materials commit verbatim — never link to external Slack/Drive/Telegram URLs; the artefact lives in-repo. (4) The planned qaherald binary (HRD-NNN to be assigned) is Herald's QA bot: it drives pherald ☐ Telegram round-trips and preserves the full conversation thread under docs/qa/qaherald-<TS>/. (5) Release gates (scripts/release.sh when implemented + the existing tag-time guard) REFUSE to tag a release whose feature-shipping commits lack their docs/qa/<run-id>/. (6) scripts/e2e_bluff_hunt.sh invariants for new features MUST cite the docs/qa/<run-id>/ artefact as their positive-evidence anchor (§11.4.2 / §11.4.5 composition).

Cascade authority. Helix Universal Constitution §11.4.83 (the verbatim operator mandate is anchored there). Herald §107.x is the project-binding restatement.

Enforcement. A feature commit that lacks docs/qa/<run-id>/ triggers FAIL at the release-gate layer. The §11.4.4 test-interrupt-on-discovery applies — the entire release cycle stops until evidence lands.

Non-compliance is a release blocker. No --qa-evidence-optional, --qa-transcript-later, --qa-bot-summary-suffices flag exists for Herald.

§107.y — Working-Tree Quiescence Rule (operator mandate, 2026-05-22; cascades from Helix §11.4.84)

Short tag: working-tree quiescence.

Forensic anchor — verbatim operator mandate (2026-05-22):

"no subagent commit may proceed while any concurrent mutation gate is in flight in the same checkout. Before `git add`, the committing agent **MUST** `grep` its own working tree for mutation markers (`MUTATED` for `paired`, `// always pass`, `return json.Marshal` shortcut paths, etc.). Any unexplained file in the staging area triggers **ABORT**."

Lesson (forensic case study — Herald-internal). On 2026-05-21 a logo-fix subagent (commit 72e81ab, "Fix: replace pandoc {width=96px} image attr with HTML tag") ran in this very checkout while a paired §1.1 Wave 4b mutation gate had temporarily injected an `// always pass` shortcut into `commons_auth/middleware.go` (JWT-bypass mutation, intended for the paired-mutation cycle to mutate → assert FAIL → restore). The logo-fix subagent's `git add` swept the mutation residue into its commit; the resulting commit was pushed to all four mirrors before any other agent caught it. Within the hour the SECURITY FIX (d5bd360, "SECURITY FIX: restore commons_auth/middleware.go JWT verify (mutation residue in 72e81ab)") restored the verify path, but the production-equivalent-binary-with-bypassed-JWT window is a real security-defect window — small, but non-zero, and demonstrably exploitable during that interval. This is no longer hypothetical; it happened. The rule below is the constitutional outcome.

Operative rule for Herald.

1. **Pre-`git add` quiescence check.** Every commit flow (main thread + subagent) **MUST** `grep` the working tree for the canonical Herald mutation markers **BEFORE** `git add`:
 - `MUTATED` for `paired` (the canonical paired-§1.1 marker emitted by `tests/test_wave4b_mutation_meta.sh`)
 - `// always pass`, `// MUTATION`, `# MUTATION` (Go + shell mutation annotations)
 - `return json.Marshal` shortcut paths in `commons` or `commons_messaging` (Wave 4b TOON mutation residue)
 - `_mutated_*` filename suffixes
 - `.git/MUTATION_IN_PROGRESS` (the lockfile — see (3))
2. **Scope-match.** Cross-check `git status --porcelain` against the subagent's declared scope. Any file outside the declared scope → **ABORT**. The subagent **MUST** explicitly account for every modified / untracked / staged entry.
3. **Lockfile serialisation.** When any mutation gate is in flight, the gate's first action is `touch .git/MUTATION_IN_PROGRESS`; its last action (in trap-on-exit) is `rm .git/MUTATION_IN_PROGRESS`. Any subagent that finds this lockfile present **MUST** refuse to `git add` and **ABORT** until the gate completes the mutate → assert-FAIL → restore → assert-PASS cycle and removes the lockfile.
4. **Worktree isolation (preferred).** When parallel subagents are required (§11.4.20 / §11.4.70 subagent-driven default), prefer `git worktree add` per subagent over single-checkout concurrency — eliminates the cross-mutation race by construction.
5. **Pre-push mutation-residue scanner.** `scripts/mutation_residue_audit.sh` (to be implemented; HRD-NNN to be assigned) **MUST** run before every push. Any commit in the pushed range containing a mutation marker → push **BLOCKED**, commit **MUST** be reverted or amended before mirrors are updated.

Prototype enforcement. `tests/test_wave4b_mutation_meta.sh` ALREADY includes a working-tree quiescence check (`check_quiescence()` at line 92; assertion at line 197 — "Working-tree quiescence — assert no MUTATED markers leaked"). It is the canonical prototype. Generalising it across all paired-§1.1 gates is open work; the universal scanner (`scripts/mutation_residue_audit.sh`) is the planned roll-out.

Cascade authority. Helix Universal Constitution §11.4.84. Herald §107.y is the project-binding restatement.

Enforcement. A mutation marker that lands in any tagged Herald commit is a critical defect regardless of how briefly it persisted — see commit 72e81ab / d5bd360 as proof.

Non-compliance is a release blocker. No `--allow-residue`, `--skip-quiescence`, `--mutation-cleanup-later` flag exists.

Mission (from the spec)

Ingesting system events and reliably fanning them out to multiple notification channels so every alert reaches the right destination without confusion.

The spec mentions "Integration into the Constitution" as a planned section — Herald is intended to plug into a larger governance/policy system that lives outside this repo. Treat that as a real constraint when designing interfaces, even though the details are TBD.

Intended stack

Go, inferred from `.gitignore (*.test, go.work, go.work.sum, coverage.*, profile.cov)`. When you scaffold, default to standard `cmd/ + internal/` layout unless the user asks otherwise, and use `go test ./.../go build ./.../go vet ./...` — there is no project-specific tooling to override these yet.

Multi-host mirror convention

`upstreams/` contains one script per mirror host:

- `GitHub.sh` — primary: `git@github.com:vasic-digital/Herald.git`
- `GitLab.sh` — `git@gitlab.com:vasic-digital/herald.git`
- `GitFlic.sh` — `git@gitflic.ru:vasic-digital/herald.git`
- `GitVerse.sh` — `git@gitverse.ru:vasic-digital/Herald.git`

Each script exports a single `UPSTREAMABLE_REPOSITORY` variable and is meant to be **sourced** (`. upstreams/GitHub.sh`), not executed for its output. The naming is intentional — capitalization matches the host's brand (`GitFlic`, `GitVerse`). Don't normalize these to lowercase or collapse them into one file; the per-host split is the interface other tooling (likely external mirror-push scripts) keys on.

When adding a new mirror, copy an existing script and change only the URL — keep the `#!/bin/bash` shebang, blank line, and `export` form identical so any consumer that greps these files keeps working.

Inheritance gate (run before any commit touching root docs)

```
bash tests/test_constitution_inheritance.sh
      # 6 invariants (I1-I6), 9 checks
bash tests/test_constitution_inheritance_meta.sh # paired §1.1
      mutation proof
```

The gate inline-walks parents for <ancestor>/constitution/Constitution.md. I5 is split into I5a–d (one check per root doc that must declare parent-discovery: CLAUDE.md, AGENTS.md, docs/guides/HERALD_CONSTITUTION.md, README.md). I6 forbids re-introducing <repo-root>/constitution/ or .gitmodules — the §104 invariant.

If either script fails, fix at root cause per Universal §11.4.4. Never silently accept the FAIL.

Spec-change rule (load-bearing — docs/specs/mvp/specification.V3.md §"Specification documents")

Any modification to a file under docs/specs/ (any depth) triggers **mandatory comprehensive planning and implementation** of the implied changes — you may not edit the spec in isolation. This rule does **not** apply to creating or renaming files; for those, ask the operator what to do with the new path. Treat every spec edit as a project-wide ripple, not a doc tweak.

Project conventions from the spec (apply when scaffolding)

These are declared in docs/specs/mvp/specification.V3.md and are easy to miss because no code enforces them yet:

- **Workable-item prefix:** HRD- (e.g. HRD-001). Use it for issues, status entries, fix logs.
- **Flavor binaries:** each Herald flavor ships as its own CLI binary, named <prefix>herald — pherald (Project Herald), sherald (System Herald), etc. Designed for CI / pipeline / cron / AI-agent invocation.
- **Layered shared code:** commons → commons_messaging (level 1) → ... → flavor. Put new shared code in the **lowest layer that still makes sense**; flavors inherit upward. commons_messaging owns the Telegram / Max / Slack / Email / Markdown-export integrations.
- **Messenger integration priority:** Telegram → Max → Slack (then Email, then Markdown/PDF/HTML export). Microsoft Teams, Lark, Discord, WhatsApp, Viber are explicitly later iterations — don't pre-implement.
- **Conversation diary:** every message in/out is appended to docs/herald/diary/main.md and re-exported to main.pdf + main.html in sync. Don't break this invariant when designing channel I/O.
- **Container stack:** Postgres (main DB) + Redis (in-memory) bundled via the containers submodule (<https://github.com/vasic-digital/containers>). All container names start with herald; all host ports start with 70XXX (70001, 70002, ...) to avoid collisions.
- **Credentials:** .env (git-ignored) with a committed .env.example. Resolution order: exported shell vars from .bashrc/.zshrc load first; .env is fallback only (never overrides shell exports). **For step-by-step setup of every supported messenger + LLM dispatcher**, see docs/guides/OPERATOR_CREDENTIALS.md — comprehensive guide covering Postgres, Redis, Telegram (HRD-011), Claude Code (HRD-012), plus

reserved env-var names for planned channels (Slack, Email, Max, Teams, ...) and dispatchers. The guide includes an audit checklist to run before every commit.

- **Vendored SDKs:** any official/unofficial messenger SDK or API client we depend on goes in as a **git submodule**, e.g. `commons_messaging/sdk/telegram` or `commons_messaging/api/telegram` — not go get'd into `go.mod`.

constitutable/ directory (parent-project extension hook)

The empty `constitutable/` directory at the repo root is intentional. Per the spec, a parent project may drop additional `Constitution.md` / `CLAUDE.md` / `AGENTS.md` (in `constitutable/`, `constitutable/<flavor>/`, `constitutable/<flavor>/<variant>/`, etc.) to layer extensions or overrides on top of the discovered `constitution/` submodule. Apply-order: `constitution/` submodule → `constitutable/` extensions → Herald's own docs. Do not delete the directory because it's empty.

Documentation artefacts (PDF/HTML siblings)

`docs/guides/HERALD_CONSTITUTION.md` and `docs/guides/CONSTITUTION_INHERITANCE.md` each ship with a committed `.pdf` sibling. When you edit one of these Markdown files, the PDF goes stale — flag it; do not regenerate silently unless the operator asks.

The HRD-lifecycle docs in `docs/` also ship as PDF/HTML/DOCX quadruples: `Issues.md` (open HRDs per V3 §8.3), `Fixed.md` (closed-HRD log per §11.4.19 atomic migration), `Status.md` (status summary), `CONTINUATION.md` (live-test handoff prompt for operator-supplied credentials). The `*_Summary.md` variants are derived; do not hand-edit.

Logo branding (added 2026-05-21). Every tracked Markdown doc now leads with a centered Herald logo header (pandoc-friendly `<div align="center">` wrapping a `` image reference to `assets/logo/herald_logo_square_128.png`). The export pipeline propagates that logo into the HTML, PDF, and DOCX siblings:

- Logo source: `assets/logo/herald_logo.png` (1664x928 RGB master). Square + transparent variants live under `assets/logo/herald_logo_square_{32,64,128,256,512,1024}.png` (chroma-keyed white → alpha). `assets/logo/herald_logo.svg` is a vector wrapper around the 512px PNG. `assets/logo/print.css` carries print/screen styling for the HTML/PDF route.
- Injection: idempotent re-runnable via `python3 scripts/branding_inject_logo.py <md ...>` — skips submodules/, containers/, `constitutable/`, `docs/diary/`, `LICENSE`; respects YAML front-matter; computes the relative path per doc depth.
- Export: `bash scripts/export_docs.sh [<md>...]` regenerates HTML (pandoc), PDF (`--pdf-engine=weasyprint`), DOCX (pandoc-native) for every `.md` that already has at least one sibling artefact committed. Pass no args to regenerate everything.
- When you add a new `.md`, run the injector once; when you edit one whose sibling exports are tracked, run the exporter for that one file.

Notes for future scaffolding

- `submodules/` holds 9 vendored Helix-stack modules (each its own `git@github.com:vasic-digital/<name>.git` repo): `auth`, `background`, `cache`, `config`, `database`, `eventbus`, `middleware`, `observability`, `recovery`. They are referenced via `replace` directives in the consuming Herald modules' `go.mod`, NOT via `go.work` (which lists the **14** Herald-owned modules — 7 foundation + `commons_auth` + 6 flavor binaries). Do not add the Helix-stack submodules to `go.work`.
- The repo is in `main` branch and committed under "Milos Vasic" — no other contributors yet.
- `.claude/` exists but is empty; project-local Claude config can go there.
- `LICENSE` is present (do not overwrite without asking).
- `.DS_Store` is now git-ignored; do not re-add the previously-stray files.